

MongoDB

Jal Irani

Adopted: Web Dev Simplified

MongoDB Introduction

- MongoDB is a noSQL database
- Incredibly easy to follow the CRUD paradigm with just a few function calls
 - Create
 - Read
 - Update
 - Delete
- Very lightweight database with not a lot of setup to hook into Node.js
- All tables (or collections) are stored as JSON (key-value) pairs
- All of it is user defined!

Terminology

- **Database:** Contains 1 or more collections. The same as a database in SQL.
- **Collection:** A group of documents inside of a database. The same as a table in SQL.
- **Document:** A single row inside of a Collection.
- **Field:** A key-value pair within a Document.

Create a Collection

```
db.createCollection("users")
```

(Creates new Collection called users)

Insert a Document into a Collection

- A document is the same as a record in SQL
- Use the insertOne() function
- The first parameter is a JSON object keys and values of each field in the document you want to insert

Using insertOne()

```
db.users.insertOne({ name: "Jal" })
```

(Creates new Document with a name of Jal into the Collection)

Using insertMany()

```
db.users.insertMany([ { name: "Jal" }, { name: "Josh" } ])
```

(Creates 2 new Documents with a name of Jal and Josh respectively into the Collection)

Querying Data From Database

- Just like using SELECT in a SQL database, use the find() or findOne() functions to find the specific
 - find() is equivalent to SELECT *
 - findOne() returns the first document

find()

- If there are no parameters, it will return an array of all key value pairs for a given collection
- You can also specify the the search on which keys you actually care about

Using find()

```
db.users.find()
```

(returns all Documents in the Collection)

```
db.users.find({ name: "Jal" })
```

```
db.users.find({ name: "Jal" }, { name: 1})
```

(returns all users with name=Jal and only returns name and _id)

```
db.users.find({}, { age: 0 })
```

(returns all users and all columns for these users except age)

Using findOne and countDocuments

```
db.users.findOne({ name: "Jal" })
```

(returns first user that matches on name=Jal)

```
db.users.countDocuments({ name: "Jal" })
```

(returns number of users with name=Jal)

sort()

- The sort() method is simple, it will return the specified query in either ascending or descending order
- The only parameter is an object to determine what order to return the result

Using sort()

```
db.users.find().sort({ name: 1, age: -1 })
```

(returns all users sorted by name in alphabetical order)

```
db.users.find().sort({ name: -1 })
```

(returns all users name in reverse alpha order)

deleteOne() and delete()

- These function will delete a document that is specified
- deleteOne() will delete the first instance found
- delete() will delete all instances found

Using deleteOne and deleteMany

```
db.users.deleteOne({ name: "Jal" })
```

(delete first user with name of Jal)

```
db.users.deleteMany({ name: "Jal" })
```

(delete all users with name of Jal)

updateOne() and updateMany()

- You can use updateOne() or updateMany() to update a given document
 - updateOne() will apply to the first instance found - and will ignore others that are found
 - updateMany() will apply to all instances
- The first parameter is the key that you are looking for to query. It can be any key, not just the first one!
- The second parameter is what you want the update result to look like
 - Note: you must match all the correct keys otherwise the keys will cease to exist

Using updateOne and updateMany

```
db.users.updateOne({ name: "Jal" }, { $set: { name:  
"Josh" } })
```

```
db.users.updateMany({ name: "Jal" }, { $inc: { name:  
"Josh" } })
```

drop()

- The drop() function will drop the specified collection
 - 2nd parameter will be either true if it worked or false if dropping the collection did not work
- dropCollection() will also give the exact same result as drop(), you can use these functions interchangeably
 - For this example, I'll just use drop() because I like brevity

Using drop()

```
db.users.drop()
```

(drop user collection, should always return true if the collection exists)

Integrating MongoDB Using Node.js

- Getting started is simple and can even use Atlas for hosting your MongoDB Database
 - Reference: <https://www.mongodb.com/languages/express-mongodb-rest-api-tutorial>